

Enterprise Advanced RAG with Hybrid Search, ReRanking, CRAG, SRAG, Caching and Guardrails in LangGraph

Github link → https://github.com/yashprogrammer/EnterpriseRAG_live.git

1. Description

An enterprise-grade conversational AI co-pilot for Site Reliability Engineers (SREs) and platform engineers. The system routes natural-language questions into a **RAG pipeline** (for documentation lookup), a **Text2SQL pipeline** (for operational data queries), or a **Hybrid pipeline** (when both are needed) — all behind a **9-layer security stack** and a **5-tier Redis cache**.

2. Duration

Stage	Approx. Time
Native RAG baseline	1.5 hours
Hybrid Search (Dense + Sparse + RRF)	1.5 hours
Cross-Encoder Reranking	1 hour
HyDE (Hypothetical Document Embeddings)	1 hour
CRAG (Corrective RAG + Web Fallback)	1.5 hours
Self-RAG (SRAG) Reflection Loop	1.5 hours
Text2SQL with LangGraph (HITL approval)	2 hours
5-Tier Caching Layer	1.5 hours
9-Layer Guardrails & Security	2 hours
Integration, FastAPI service, Streamlit demo	1.5 hours
Total	≈ 15–17 hours

3. Introduction

Retrieval-Augmented Generation (RAG) has become the de-facto pattern for grounding Large Language Models in private enterprise knowledge. However, a *naïve* RAG pipeline — embed → retrieve top-k → stuff into the prompt → generate — collapses the moment it meets the real world: noisy corpora, ambiguous queries, latency-sensitive users, prompt-injection attempts, and the need to also consult structured operational data.

This project builds a **production-ready RAG copilot for Kubernetes operations** that progressively layers in every advanced RAG technique published over the past two years — Hybrid Search, Reranking, HyDE, CRAG, Self-RAG — and combines them with **Text2SQL**, **multi-tier caching**, and a **9-layer security pipeline**. The knowledge base is deliberately engineered with a **95% noise / 5% signal** ratio so each advanced technique has to *earn its place*. The entire flow is orchestrated as a stateful, resumable **LangGraph** state machine with **human-in-the-loop SQL approval**.

4. Aim

Goal: Build a working, end-to-end, production-grade RAG service that an SRE can actually trust with operational queries — accurate, fast, observable, and safe against prompt injection.

Specifically the learner will:

1. Understand how each advanced RAG technique solves a *specific* failure mode of naïve RAG.
2. Implement Dense + Sparse hybrid retrieval with **Reciprocal Rank Fusion**.
3. Build a **LangGraph** state machine with conditional edges, persistence, and **interrupts** for human approval.
4. Integrate **Text2SQL** alongside RAG behind a single intelligent router.
5. Engineer a **5-tier Redis cache** that bounds latency and LLM token spend.
6. Wire up a **9-layer defensive security pipeline** (input validation, prompt-injection scanning, PII redaction, output validation).

7. Ship the whole thing as a clean, testable **FastAPI** service with Docker and a Streamlit demo UI.

5. Dataset Used

The knowledge base is assembled by `scripts/data_pipeline/` and is intentionally adversarial.

Category	Source	Format	Count	Size
Signal (true_data)	Kubernetes official docs — kubernetes.io	PDF / MD	~50 docs	~30 MB
Noise (noisy_data)	Random PDFs from github.com/tpn/pdfs	PDF / DOCX / TXT	~950 docs	~120 MB
Structured Ops DB	Synthetic K8s operational data	PostgreSQL (7 tables)	~10k rows	~20 MB

Why 95% noise? A 95/5 noise-to-signal ratio guarantees that naïve top-k retrieval *will* fail — which is exactly the pedagogical point. Each advanced technique then has to demonstrably rescue the signal.

The synthetic SQL operational schema:

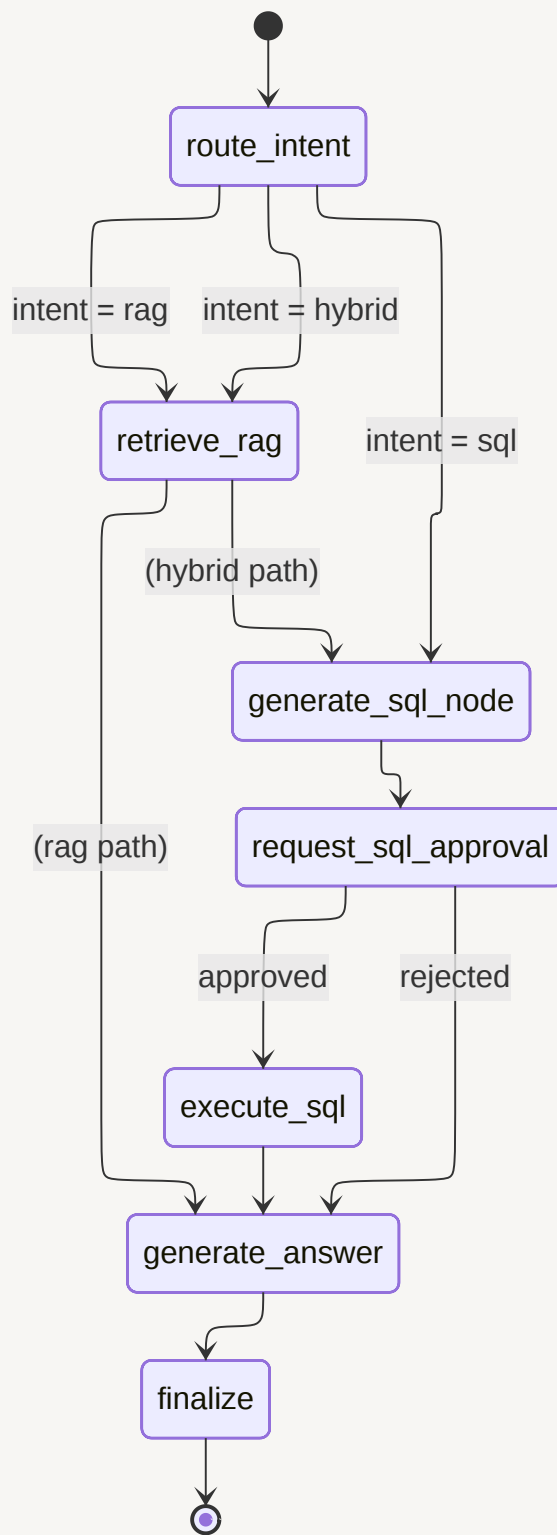
```
clusters(id, name, region, provider, k8s_version, node_count, status, created_at)
nodes(id, cluster_id, name, role, instance_type, cpu_cores, memory_gb, status, joined_at)
pods(id, node_id, namespace, name, image, cpu_request, memory_request, status, created_at, last_restart)
deployments(id, cluster_id, namespace, name, replicas_desired, replicas_ready, strategy, updated_at)
incidents(id, cluster_id, severity, title, status, started_at, resolved_at, mttr_minutes)
alerts(id, cluster_id, node_id, alert_name, severity, fired_at, resolved_at, labels JSONB)
oncall_logs(id, incident_id, engineer, action, notes, logged_at)
```

6. Tools & Technologies

Layer	Technology
Language	Python 3.12
API Framework	FastAPI 0.115
Orchestration	LangGraph (with Postgres checkpointing + interrupts)
LLM	OpenAI GPT-4o / GPT-4o-mini
Embeddings	OpenAI <code>text-embedding-3-small</code>
Vector Store	Qdrant (dense + sparse vectors)
Sparse Encoder	FastEmbed / BM25
Reranker	Local cross-encoder (BGE) / Voyage AI
Relational DB	PostgreSQL 16
Cache	Upstash Redis (serverless)
Web Search Fallback	Tavily Search API
Security Scanner	<code>llm-guard</code> (PromptInjection, Toxicity, BanTopics, Anonymize)
Document Parsing	Docling / pypdf
Auth	JWT (PyJWT) + sliding-window rate limiter
Demo UI	Streamlit
Container	Docker + Docker Compose
Deployment	AWS ECS Fargate + EFS + ALB (CloudFormation)
Logging	Loguru (structured JSON)
Testing	pytest, Ragas (eval harness)
Tooling	uv (deps), Ruff (lint), Mypy (types)

7. Architecture Diagram

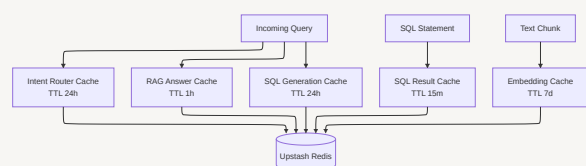
7.1 High-Level System Architecture



7.3 RAG Retrieval Pipeline (HyDE → Hybrid → Rerank → CRAG → Spotlight → Generate)



7.4 Caching Topology



8. Key Steps / Modules

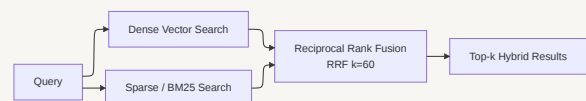
The project is built **incrementally** — each commit adds one self-contained advanced technique on top of the previous baseline. Learners can checkout any commit and run that exact stage.

Module 1 — Native (Baseline) RAG

Set up Qdrant, embed the Kubernetes docs, retrieve top-k, stuff into the prompt, generate. The intentional weakness here — naïve dense retrieval drowning in 95% noise — is what motivates everything that follows.

Module 2 — Hybrid Search (Dense + Sparse + RRF)

Add a **sparse encoder** (BM25-style) running side-by-side with dense embeddings inside Qdrant, fused with **Reciprocal Rank Fusion**. Dense captures *semantic similarity* ("rolling update" ≈ "progressive rollout"), sparse rescues *exact lexical matches* ("kubectll", "CrashLoopBackOff").



Module 3 — Cross-Encoder Reranking

Bi-encoder retrieval is *fast but shallow*. A **cross-encoder** re-scores the top-N candidates by jointly encoding the (query, doc) pair — slower per-pair, but dramatically better at separating signal from noise. Pluggable backend: local BGE model or Voyage AI.

Module 4 — HyDE (Hypothetical Document Embeddings)

Short SRE queries like *"why is my pod OOMKilled?"* lack vocabulary overlap with longer documentation chunks. **HyDE** asks an LLM to generate 3 *hypothetical answers*, embeds those, and uses them as the retrieval query — bridging the vocabulary gap.

Module 5 — CRAG (Corrective RAG + Web Fallback)

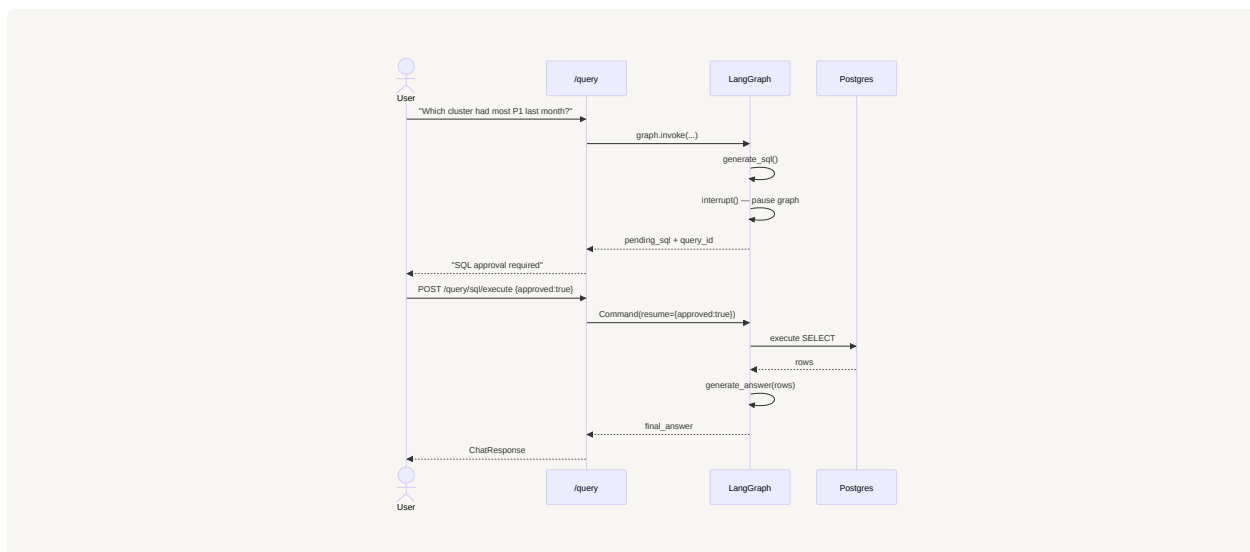
A grader LLM scores each retrieved chunk for relevance. If the average score falls below `CRAG_RELEVANCE_THRESHOLD = 0.7`, the system falls back to **Tavily web search** — turning retrieval failure into graceful degradation instead of a hallucination.

Module 6 — Self-RAG (SRAG) Reflection Loop

After the LLM generates an answer, a **reflection critic** scores it. If the score is below `REFLECTION_MIN_SCORE = 0.8`, the graph loops back and regenerates (max 2 retries). Also teaches the system *when retrieval isn't needed at all* — general K8s knowledge bypasses the retriever entirely.

Module 7 — Text2SQL with LangGraph (Human-in-the-Loop)

Not every question is documentation — *"Which cluster had the most P1 incidents last month?"* needs **SQL, not RAG**. An LLM generates SQL from the Postgres schema, the LangGraph **interrupts** to surface the SQL for human approval, then resumes execution via `Command(resume={approved})`. SELECT-only enforcement + keyword blocklists.



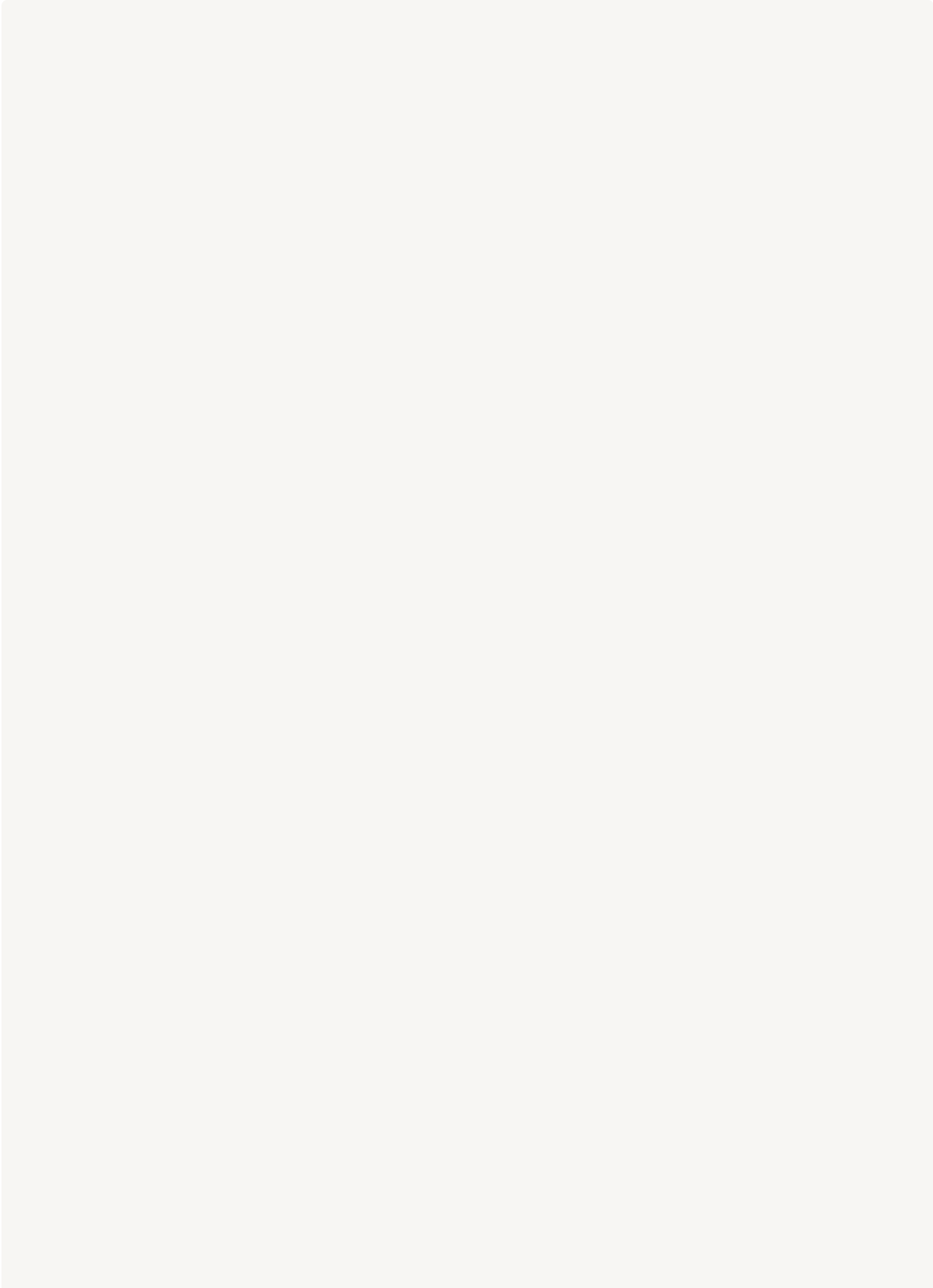
Module 8 — 5-Tier Caching Layer

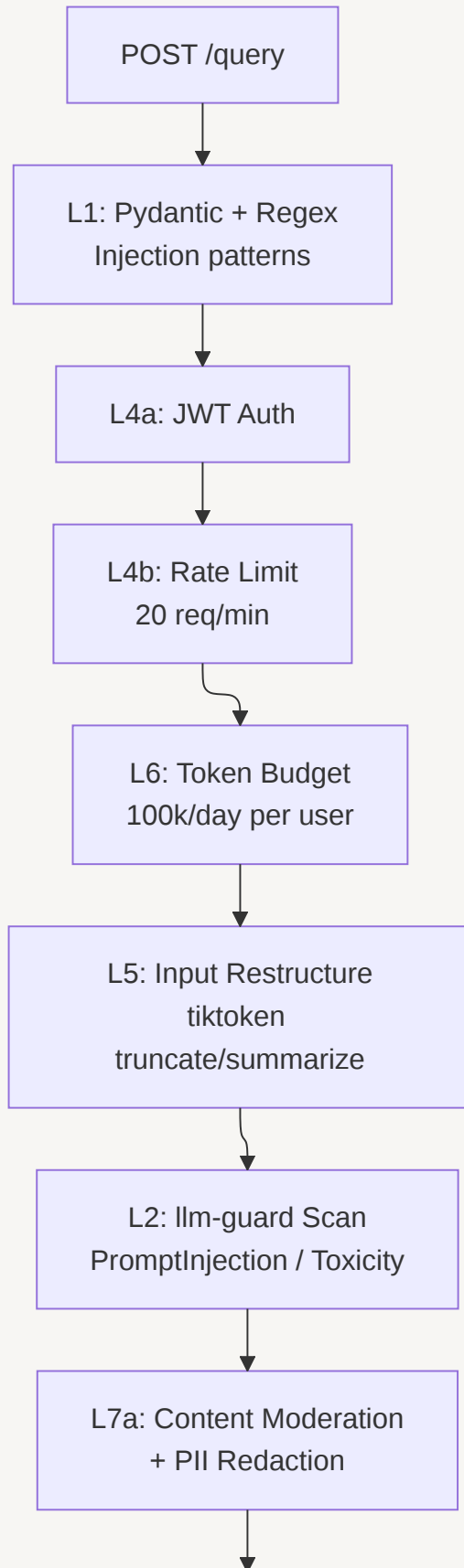
Every expensive call (embedding, intent classification, SQL generation, SQL execution, final answer) is wrapped with a SHA-256-keyed Upstash Redis cache. Cache hits are surfaced in the response metadata for full observability.

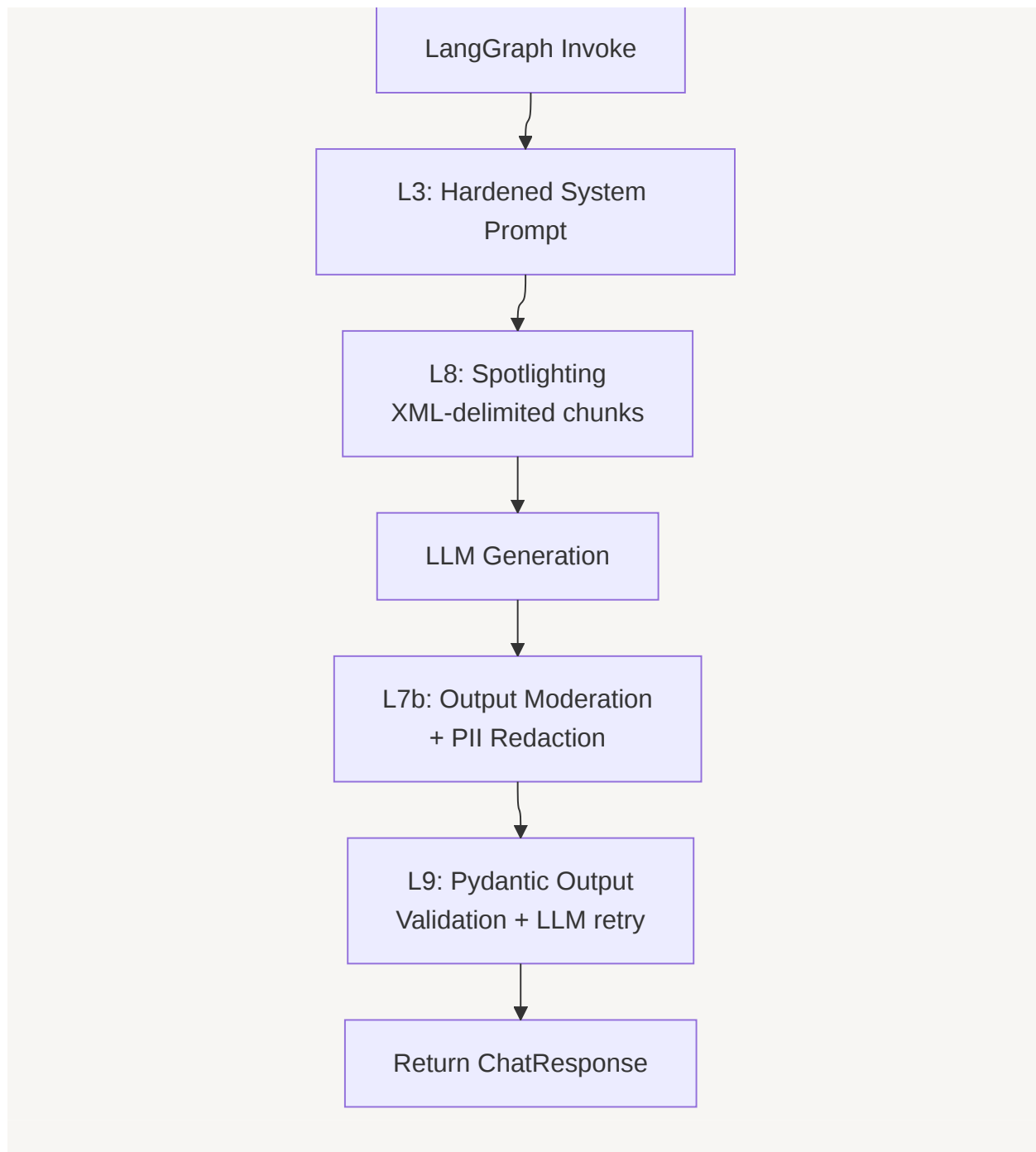
Tier	Key	TTL	Saves
embedding	sha256(text)	7 days	Embedding API calls
intent_router	sha256(question.lower())	24 h	Router LLM calls
sql_gen	sha256(question)	24 h	SQL-generation LLM calls
sql_result	sha256(normalized SQL)	15 min	Postgres reads
rag_answer	sha256(question + flags)	1 h	Full pipeline cost

Module 9 — 9-Layer Guardrails & Security

Every request traverses a fixed-order defensive pipeline. **Defense in depth** — no single layer is trusted to stop everything.







9. Learning Outcomes

After completing this project, learners will be able to:

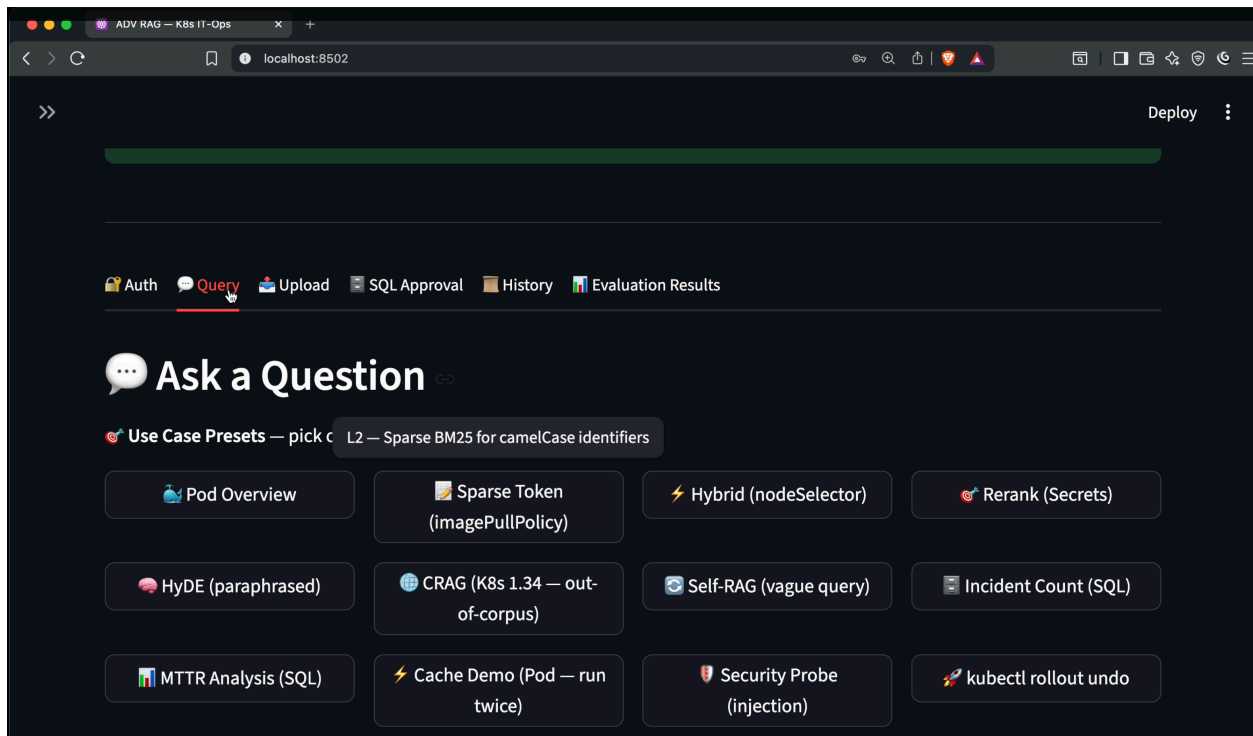
- **Diagnose** the specific failure modes of naïve RAG (vocabulary mismatch, noisy retrieval, hallucination, context-window flooding) and pick the right technique to

fix each one.

- **Implement Hybrid Search** with Qdrant — dense + sparse vectors fused via Reciprocal Rank Fusion.
 - **Build a cross-encoder reranking pipeline** with pluggable backends (local BGE / Voyage AI).
 - **Apply HyDE** to bridge the query-document vocabulary gap with hypothetical-answer embeddings.
 - **Engineer Corrective RAG (CRAG)** with relevance grading + Tavily web-search fallback.
 - **Implement Self-RAG** — reflection loops that decide *whether to retrieve at all* and *whether to regenerate*.
 - **Design and code a LangGraph state machine** with conditional edges, Postgres checkpointing, and `interrupt()` based human-in-the-loop approvals.
 - **Build a safe Text2SQL system** — LLM SQL generation, SELECT-only enforcement, keyword blocklists, and HITL approval before execution.
 - **Engineer a multi-tier Redis cache** keyed by content hashes, with TTLs tuned per workload.
 - **Wire up 9 defensive security layers** — Pydantic validation, JWT, rate-limit, token budgets, prompt-injection scanning, PII redaction, spotlighting, structured-output validation with LLM retry.
 - **Package a FastAPI service** with Docker Compose, JWT auth, and a Streamlit demo UI.
 - **Evaluate a RAG system** with Ragas on a held-out question set.
-

10. Screenshots

Streamlit Demo UI



11. Optional Add-ons

Possible enhancements learners can attempt after finishing the core project:

- **Multi-LLM Provider Support** — Plug in Anthropic Claude (Opus / Sonnet / Haiku), Google Gemini, or local Llama via Ollama behind a provider-agnostic interface.
- **Multi-Modal RAG** — Extend ingestion to embed page-level images and tables (Docling already supports this) and serve them in answers.
- **Observability Stack** — Add **Langfuse** or **Arize Phoenix** for distributed tracing of every LLM call, retrieval, and cache hit
- **Multi-Lingual Support** — Replace `text-embedding-3-small` with a multilingual model (e.g. BGE-M3) and add language detection at the router.
- **Streaming Responses** — Convert `/query` to a streaming SSE endpoint and stream tokens to the Streamlit UI.

12. Resource Links

Core Papers

- **HyDE** — *Precise Zero-Shot Dense Retrieval without Relevance Labels* — arxiv.org/abs/2212.10496
- **CRAG** — *Corrective Retrieval Augmented Generation* — arxiv.org/abs/2401.15884
- **Self-RAG** — *Learning to Retrieve, Generate, and Critique through Self-Reflection* — arxiv.org/abs/2310.11511
- **Reciprocal Rank Fusion (RRF)** — Cormack et al. 2009 — plg.uwaterloo.ca/~gvcormac/cormacksigir09-rrf.pdf
- **Spotlighting / Prompt-Injection Defenses** — *Defending Against Indirect Prompt Injection Attacks With Spotlighting* — arxiv.org/abs/2403.14720

Official Documentation

- **LangGraph** — langchain-ai.github.io/langgraph/
- **LangChain** — python.langchain.com
- **FastAPI** — fastapi.tiangolo.com
- **Qdrant** — qdrant.tech/documentation
- **Upstash Redis** — upstash.com/docs/redis
- **OpenAI API** — platform.openai.com/docs
- **Tavily Search** — docs.tavily.com
- **llm-guard** — llm-guard.com
- **Docling** — github.com/DS4SD/docling
- **Ragas (Eval)** — docs.ragas.io

Helpful Watch / Read

- LangGraph Human-in-the-Loop tutorial — langchain-ai.github.io/langgraph/concepts/human_in_the_loop/
- Qdrant Hybrid Search Guide — qdrant.tech/articles/hybrid-search/

- OWASP Top 10 for LLM Applications — genai.owasp.org
-